



Programação Orientada a Objetos em Java

Um guia prático para dominar os conceitos fundamentais da programação orientada a objetos através da criação de um sistema de livros.

O que vamos aprender hoje?

1. Fundamentos da Orientação a Objetos

- Classes: o molde dos objetos
- Instanciação: moldes em objetos reais
- Métodos: ações e comportamentos
- Estado do objeto: onde os dados vivem

2. O Ciclo de Vida do Objeto

- Construtores: nascendo com os dados certos
- Dependências: tudo o que o objeto precisa para funcionar

3. Pilar: Encapsulamento

- Modificadores de acesso: protegendo os dados
- Isolamento de comportamentos: cada classe cuida da sua lógica
- Getters e setters: acesso seguro às informações

4. Arquitetura e Boas Práticas

- Vantagens da POO: mais clareza e reutilização
- Organização de código: projetos limpos e sustentáveis

Vantagens da Orientação a Objetos



Reutilização de código

Classes podem ser reaproveitadas em múltiplos projetos, economizando tempo e esforço de desenvolvimento.



Modularidade

Divide o sistema em componentes independentes que podem ser desenvolvidos e testados separadamente.



Mantabilidade

Facilita a manutenção e extensão do código à medida que o sistema cresce em complexidade.



Orientação a Objetos: Classe

Criando um molde de livros

Uma classe define o molde que todos os objetos seguirão, especificando atributos como título, autor e preço.

Objetos para todos os lados!

Cada instância de Livro é um objeto único com seus próprios valores de atributos.

Criando um novo método

Métodos adicionam comportamentos à classe, como calcular desconto ou exibir informações do livro.

Entendendo a construção

Construtores inicializam objetos com valores específicos ao momento da criação.

Orientação a Objetos: Método

Criando um novo método

Métodos adicionam comportamentos à classe, como calcular desconto ou exibir informações do livro.

Entendendo a construção

Construtores inicializam objetos com valores específicos ao momento da criação.

Orientação a Objetos: Classe

```
public class Livro {  
  
}
```

Orientação a Objetos: Classe e atributos

```
public class Livro {  
    String nome;  
    String descricao;  
    double valor;  
    String isbn;  
}
```

Orientação a Objetos: Instanciação de um objeto

```
Livro livro = new Livro();
```


Orientação a Objetos: Instanciação de um objeto

```
Livro livro = new Livro();  
livro.nome = "A revolução dos bichos";  
livro.descricao = "Verdadeiro clássico moderno, de um dos mais influentes escritores do século XX";  
livro.valor = 59.90;  
livro.isbn = "978-85-66250-46-6"
```

Orientação a Objetos: Método

```
tipoDeRetorno nomeDoComportamento() {  
  
    //Código que será executado.  
  
}
```

Orientação a Objetos: Método

```
void mostrarDetalhes() {  
    System.out.println(nome);  
    System.out.println(descricao);  
    System.out.println(valor);  
    System.out.println(isbn);  
}
```

Orientação a Objetos: Escopo de métodos.

```
void mostrarDetalhes() {  
    String mensagem = "Mostrando detalhes do livro";  
    System.out.println(mensagem);  
    System.out.println(nome);  
    System.out.println(descricao);  
    System.out.println(valor);  
    System.out.println(isbn);  
}
```

Orientação a Objetos: Composição

É um conceito fundamental de Orientação a Objetos onde uma classe (o "todo") contém referências a instâncias de outras classes (as "partes") como seus atributos.

O que aconteceria se precisássemos representar o Autor do livro, que contém as seguintes características?
Nome, email e cpf.

Autor passa a ser um atributo de Livro, que o compõe.

```
public class Livro {  
    String nome;  
    String descricao;  
    double valor;  
    String isbn;  
    Autor autor;  
}
```

Orientação a Objetos: Métodos com parâmetros.

Vamos criar um método para calcular um possível desconto no livro.

O desconto precisa ser passado para o comportamento.

Atenção ao this.

Orientação a Objetos: Métodos com retorno.

Nossa classe Livro já possui dois métodos, mas os dois são void. Vimos que o void representa a ausência de um retorno, mas há situações em que precisaremos retornar algo.

Vamos criar um método para verificar se há autor cadastrado para o Livro.

Encapsulamento: Protegendo Nossos Dados

01

Isolando comportamentos

Cada classe gerencia sua própria lógica sem expor detalhes internos desnecessários.

Getters e Setters

Métodos públicos que controlam o acesso aos atributos privados, permitindo validação e lógica adicional.

02

Código encapsulado

Atributos privados garantem que mudanças internas não quebrem o código externo.

Construtor com dependências

Define quais componentes o objeto precisa no momento da criação, estabelecendo contratos claros.

Vamos nos divertir!

Kahoot!

www.kahoot.it